🚀 Migrating On-Prem Oracle Database to AWS RDS using Data Pump & S3 Integration

Migrating enterprise databases from **on-premises to the cloud** is one of the most critical steps in modernization. Oracle workloads, being mission-critical, need a **secure, scalable, and low-risk migration path**.

There are multiple options:

- ✅ **Oracle SQL Developer** → simple, UI-driven imports for small databases (<50 MB).
- ✅ **AWS DMS (Database Migration Service)** → good for heterogeneous migrations and **ongoing replication**.
- ✅ **Oracle Data Pump with Amazon S3** → the **preferred choice for large/complex databases** (hundreds of MBs to TBs).

In this post, I'll walk through the **theory + hands-on steps** to migrate an **on-premise Oracle schema** to **Amazon RDS for Oracle** using **Oracle Data Pump + Amazon S3 integration**.

✅ Best Practices for Oracle Data Pump on RDS

- **Use schema or table mode** → Amazon RDS does not support FULL imports (SYS/SYSDBA objects are restricted).
- **Import only required schemas** → reduces risk & migration time.
- **Avoid unsupported parameters** → TRANSPORT_TABLESPACES, TRANSPORTABLE, TRANSPORT_FULL_CHECK.
- **Pre-create tablespaces in RDS** → remap if needed since custom tablespaces may not exist in RDS.
- **Ensure S3 bucket is in the same AWS Region** as your RDS instance.
- **Validate storage availability** on both source and target.
- **Use parallelism** → speeds up large migrations.

📦 Pre-Requisites with Amazon S3

- Create an **Amazon S3 bucket** in the **same region as RDS**.
- Enable **RDS–S3 integration** (requires IAM role with proper permissions).
- Ensure your **IAM policy** allows read/write between RDS and S3.
- Verify **dump file size** does not exceed allocated storage.

This integration is what makes the migration **fast and seamless**.

⚙️ Migration Workflow

1️⃣ Export Schema from On-Prem DB

Data Pump export creates a **binary dump file** containing schema objects, metadata, and data.

```
expdp system/password DIRECTORY=DATA_PUMP_DIR \
DUMPFILE=appuser_onprem.dmp \
LOGFILE=expdp_appuser_onprem.log \
SCHEMAS=APPUSER \
FLASHBACK_TIME=SYSTIMESTAMP
```

⚡ Why Flashback Time? → Ensures a **consistent snapshot** of the database during export.

2️⃣ Prepare RDS Environment

Create users, assign default tablespaces, and grant required privileges.

```
CREATE USER appuser IDENTIFIED BY "appuser12345" DEFAULT TABLESPACE users;

GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO appuser;
```

⚡ Why? → RDS does not allow SYSDBA access, so **schemas must be created manually**.

3️⃣ Transfer Dump to RDS via S3

Upload the .dmp file to **Amazon S3**, then pull it into the RDS instance.

```
SELECT rdsadmin.rdsadmin_s3_tasks.download_from_s3 (
  p_bucket_name    => 'infras3',
  p_s3_prefix      => 'APPUSER_ONPREM.DMP',
  p_directory_name  => 'DATA_PUMP_DIR'
) task_id
FROM dual;
```

⚡ Why? → RDS does not allow OS-level access, so S3 acts as a **secure staging area**.

Check logs:

```
SELECT text FROM table(rdsadmin.rds_file_util.read_text_file('BDUMP','dbtask-xxxx.log'));
```

4️⃣ Import Schema into RDS using Data Pump API

```
DECLARE
 v_hdl NUMBER;
BEGIN
 v_hdl := DBMS_DATAPUMP.OPEN(operation => 'IMPORT', job_mode => 'SCHEMA');

 DBMS_DATAPUMP.ADD_FILE(v_hdl,'APPUSER_ONPREM.DMP','DATA_PUMP_DIR',
                dbms_datapump.ku$_file_type_dump_file);
 DBMS_DATAPUMP.ADD_FILE(v_hdl,'impdp_appuser_onprem.log','DATA_PUMP_DIR',
                dbms_datapump.ku$_file_type_log_file);

 DBMS_DATAPUMP.METADATA_FILTER(v_hdl,'SCHEMA_EXPR','IN (''APPUSER'')');
 DBMS_DATAPUMP.METADATA_REMAP(v_hdl,'REMAP_TABLESPACE','APP_TBS','USERS');

 DBMS_DATAPUMP.SET_PARALLEL(v_hdl,2);
 DBMS_DATAPUMP.START_JOB(v_hdl);
END;
/
```

⚡ Key points:

- **Metadata filter** → restricts import to the target schema.
- **Remap tablespaces** → maps on-prem tablespaces to RDS-supported ones.
- **Parallelism** → speeds up the import process.

5️⃣ Validate Import

- Check Data Pump log file:

```
SELECT text FROM
table(rdsadmin.rds_file_util.read_text_file('DATA_PUMP_DIR','impdp_appuser_onprem.log'));
```

- Verify schema & objects:

```
SELECT * FROM dba_users WHERE username='APPUSER';

SELECT object_name, object_type FROM dba_objects WHERE owner='APPUSER';
```

⚡ Validation ensures **data integrity** and confirms migration success.

🔎 Key Takeaways

👉 **Oracle Data Pump with Amazon S3 integration** is a **robust, fast, and secure method** for migrating large Oracle databases to **Amazon RDS**.

It is ideal when:

- You're dealing with **large datasets**.
- You need **parallelized, high-performance imports**.
- You want **fine-grained control** over schemas and tablespaces.

For **real-time replication** or **minimal downtime**, consider **AWS DMS** or **Oracle GoldenGate**.